Compiler Register Allocation

DA Programming Project II

Group T9G2 - Spring 2026

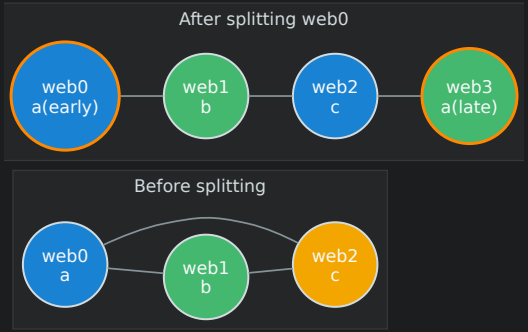
Alexandre Dinis Alves Teixeira
Carlos Francisco de Sousa Ferreira Magalhaes Diogo
Rodrigo Martins Dias

`make test` `make docs` `./register_alloc -b ...`

Pipeline

Core flow

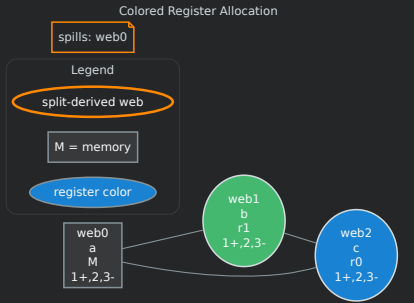
- parse ranges + config
- fuse ranges into webs
- build the interference graph
- color with at most K registers
- export allocation text and colored DOT



Doxygen output in `docs/html` documents the graph model, algorithms, and complexity analysis.

Spilling and Splitting

Bounded spilling



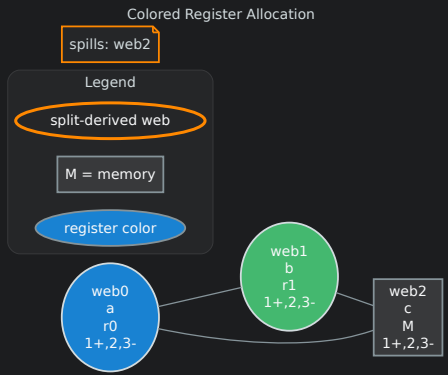
Bounded splitting

Metadata remains parseable: `spills: N`, `spill: webX`, `splits: N`, `split: webX -> webY,webZ`.

Custom Free Algorithm

Pressure-aware choice

- count distinct neighbor register colors
- choose the web with the highest count
- break ties by highest graph degree
- assign the lowest compatible register
- use memory only when all registers conflict



Demo Inputs and Commands

Advanced examples

- complex_allocation.txt
- free4.txt
- spilling5.txt
- splitting_showcase.txt
- splitting2.txt

```
make test
make docs
./register_alloc -b
inputs/advanced/ranges/
complex_allocation.txt
inputs/advanced/registers/free4.txt
advanced_free.txt
```

The dense input shows fused webs, high pressure, register reuse, and memory assignments.

Complexities

Build webs: $O(V \cdot R^2 \cdot P \log P)$
Build graph: $O(W^2 \cdot P + E \cdot W)$
Basic: $O(W \cdot (W^2 + E))$

Spilling: $O((S+1) \cdot W \cdot (W^2 + E))$
Splitting: $O((S+1) \cdot (W^2 \cdot P + E \cdot W + W \cdot (W^2 + E)))$
Free: $O(W \cdot (W^2 + E))$

`W` webs, `E` directed adjacency entries, `P` program points, `R` ranges per variable, `S` recovery bound. Bounds include the vector-backed `Graph<int>::findVertex` cost.